

Modality-Driven Search with Holistic Trace Judging for ARC-AGI-2

Johan Land
Independent Researcher
johan.land@gmail.com

Abstract—Large language models can produce fluent, internally coherent reasoning traces for abstract reasoning tasks while still being confidently wrong — making selection among candidates, not just generation, the central challenge. I present a solver for ARC-AGI-2 (a few-shot visual reasoning benchmark) built around two principles: (i) treating reasoning modalities as search operators, generating diverse candidates independently across text, image, and code channels, and (ii) context-preserving holistic judging, in which a judge model jointly compares all candidate reasoning traces within a single long-context prompt. Unlike self-consistency or majority voting, this approach reliably recovers correct minority hypotheses on tasks where the modal answer is wrong.

On the ARC Prize semi-private evaluation set, the solver achieves 72.9% at \$38.99/task — the highest score on the verified leaderboard at the time of writing, exceeding the best standalone frontier models (GPT-5.2 Pro at 54.2%, Gemini 3 Pro at 54.0%) by +18.7 percentage points. On the public evaluation set, it achieves 76.1% at \$19.69/task. I release the full source code and document extensive negative results, including the finding that prescriptive prompting templates and iterative refinement systematically reduce hypothesis diversity and degrade performance.

Index Terms—ARC, ARC-AGI-2, large language models, multimodal reasoning, program synthesis, LLM-as-a-judge

I. INTRODUCTION

A central challenge in applying LLMs to abstract reasoning is not just producing candidate solutions, but **knowing what is right and what is wrong** in a setting where models can be confidently incorrect—even when they provide detailed, plausible reasoning traces.

ARC-AGI-2 was designed to be *easy for humans and hard for AI*, and—critically—to measure both **capability** and **efficiency** (cost). Progress on ARC-style benchmarks has been rapid: ARC Prize reports significant year-over-year improvements driven by frontier reasoning systems and application-layer refinement harnesses [1].

This paper describes an ARC-AGI-2 solver that treats **modalities as search operators** and uses **judging as the final selection mechanism**: generate diverse candidate solutions across independent reasoning channels, then select among them using context-preserving holistic judging.¹

¹An extended version of this paper with complete failure analysis, task-level breakdowns, judge rationale transcripts, and additional ablation details is available as a preprint [2].

A. Contributions

- **A modality-driven search solver for ARC-AGI-2** that generates candidates independently across text, image, and code reasoning channels.
- **A context-preserving holistic judge in this setting** that reads all candidate traces jointly to select the best outputs, identifying correct *minority* hypotheses—yielding +7 solved instances over majority vote at only 13% of total system cost (Section V).
- **Verified ARC-AGI-2 semi-private performance: 72.9%** at \$38.99/task on the ARC Prize Verified leaderboard²—the highest score on the leaderboard at the time of writing, exceeding the best standalone frontier models (GPT-5.2 Pro at 54.2%, Gemini 3 Pro at 54.0%) by +18.7 percentage points.
- **Public eval performance: 76.11%** at \$19.69/task (self-measured).
- **Open-source release** of the full source code, detailed negative results, and complete public-evaluation run data to support future research.

II. BACKGROUND AND RELATED WORK

The Abstraction and Reasoning Corpus (ARC) was introduced by [3] as a benchmark for measuring intelligence as skill-acquisition efficiency—how effectively a system acquires new skills under constrained experience and priors. Tasks are framed as few-shot input–output induction over grids: given a handful of training demonstration pairs, the solver must infer an underlying transformation rule and apply it to a held-out test input. ARC-AGI-2 [1] extends the original benchmark with harder tasks, formalized dataset splits (training, public, semi-private, and private evaluation sets of calibrated tasks), pass@2 scoring, first-party human calibration, and an emphasis on both accuracy and cost efficiency. Fig. 2 illustrates a representative task.

Classical ARC solvers treat tasks as latent programs composed from a hand-designed DSL [4]. Learned approaches attack ARC via induction or transduction; [5] show that ensembling both paradigms can approach human-level performance. Test-time adaptation has emerged as a major axis: [6] demonstrate that updating model parameters at test time yields

²<https://arcprize.org/leaderboard>

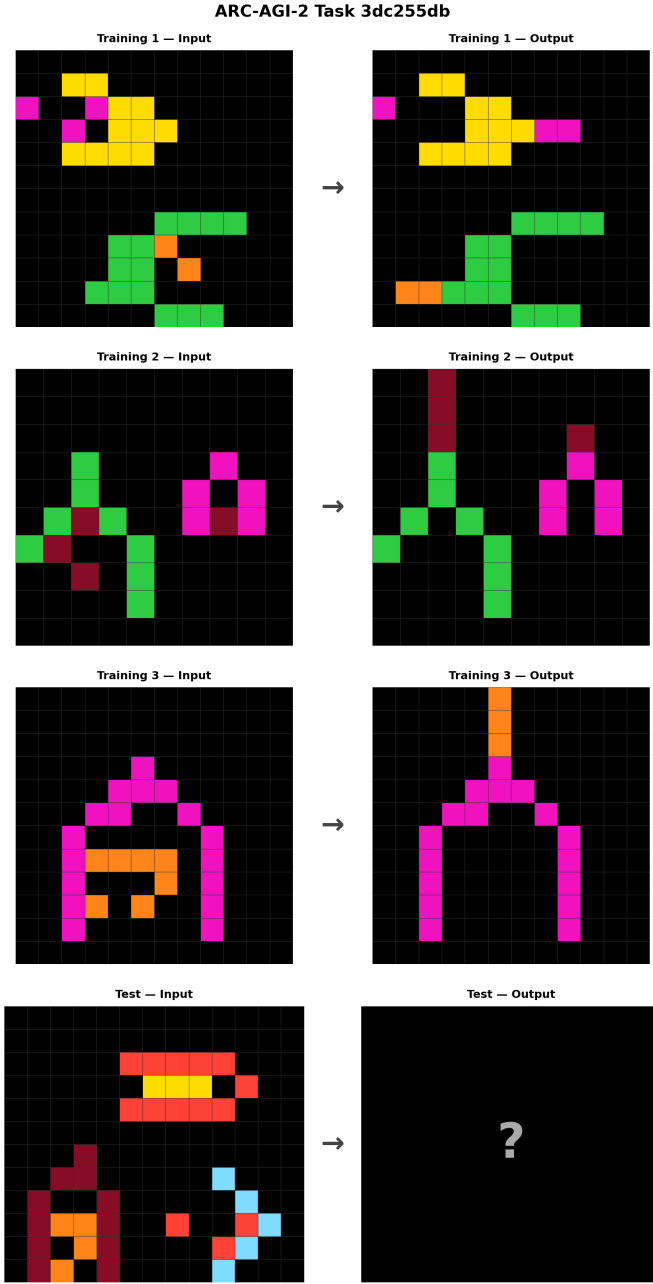


Fig. 1. ARC-AGI-2 task 3dc255db. A human might see “spaceships” with particles on the exhaust side. The transformation removes the particles from the interior and extends them from the nose. Three training pairs (rows 1–3) demonstrate the rule; the test input (row 4) must be solved from these examples alone. This task remains unsolved by the system.

strong gains, while [7] combine learned representations with explicit program search.

The broader LLM literature has developed test-time compute techniques directly relevant to this work. Chain-of-thought prompting [8] elicits intermediate reasoning steps; self-consistency [9] samples multiple paths and selects the most common answer. Tree of Thoughts [10] reframes inference as search with branching and backtracking. ReAct [11] interleaves reasoning with tool calls, and PAL [12] translates problems into executable code. Iterative-refinement methods [13], [14] improve outputs through feedback loops but risk anchoring to early hypotheses. [15] show that optimally allocating test-time compute can be more effective than scaling model parameters.

Relative to this landscape, the present work composes three trends: (i) hypothesis generation as explicit search across *multiple reasoning modalities*, extending ToT-style inference [10]; (ii) tool- and program-mediated reasoning [11], [12] used to produce richer intermediate artifacts for downstream selection; and (iii) judge-based selection adapted from the LLM-as-a-judge paradigm [16]. What is distinctive is the combination of heterogeneous candidate generators with context-preserving comparison over full reasoning traces, rather than scalar scoring or consensus compression.

III. METHOD

The solver is built around a practical observation: to solve tasks that frontier systems do not already solve, the correct solution is often a **minority hypothesis**. If the most common solution were correct, the task would likely already be within the main cluster of model behavior. The solver’s first phase therefore creates many *independent* candidate solutions across heterogeneous reasoning modalities (text, image, and code), maximizing the probability that at least one candidate captures a genuinely novel hypothesis. Each modality provides a structurally different representation of the same task, which empirically produces candidates that cluster differently (Section IV-E).

A. Pipeline Overview

Fig. 2 shows the end-to-end pipeline. For each ARC task:

1. **Candidate generation (up to 29 candidates).** A set of modality-specific solvers is run, each producing a predicted output grid, a reasoning trace, and, for code candidates, the code plus tool-use logs. Each candidate produces one output; the pass@2 two-guess format is introduced only at the judging stage. Candidate generation proceeds in stages: if early candidates already show strong agreement, the system terminates early and skips the remaining, more expensive modalities. This adaptive early stopping improves cost efficiency on tasks that can be solved with a shallower search. On the ARC-AGI-2 public evaluation, 37 of 167 instances stopped after the initial 8-candidate probe, and **36 of those 37**

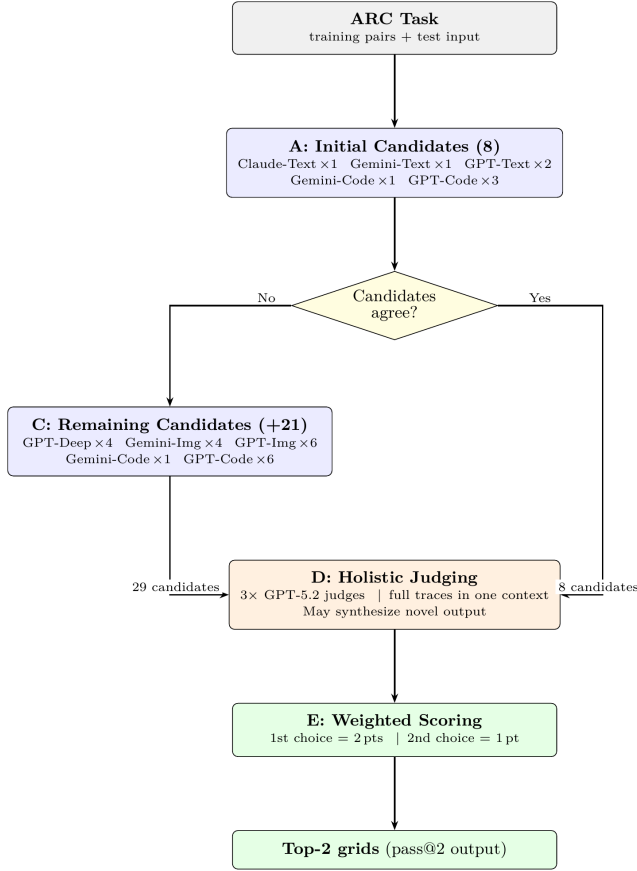


Fig. 2. System pipeline overview.

were solved correctly (97.3%), showing that shallow search is often sufficient on easier tasks. The same principle is even more effective on easier ARC-AGI-1 tasks, where the system achieves 94.5% on the official semi-private evaluation at only \$11.40/task, substantially cheaper than the \$38.99/task on ARC-AGI-2. Both semi-private scores and costs are as reported by ARC Prize’s verification infrastructure. The single early-stopping failure (dbff022c:1) is a case of extreme groupthink: all models confidently assume the simpler of two valid legend-to-color interpretations while the ground truth requires the more complex one. This validates the heuristic on easy tasks, but also shows that early consensus can mask genuine ambiguity.

2. **Holistic judging (3 parallel judges).** All candidate traces are concatenated into a single long-context prompt. Each judge model is asked to identify the top-2 most likely correct candidates (or propose a synthesis that recombines correct subcomponents), explain why other candidate clusters are wrong, and output the final grids.
3. **Weighted scoring.** Each judge’s first choice receives 2 points and second choice receives 1 point. The system sums points across judges for each distinct output grid.

The two grids with the highest total score become the solver’s pass@2 guesses.

ARC-AGI-2 tasks often introduce new concepts that were not exercised in the training pairs. A solution can be perfectly “logical” on the training examples but still be a brittle overfit that fails to abstract the intended rule. This makes naive logic checking of traces insufficient (Sections III-C and VI). In practice, the system needs broad hypothesis exploration across structurally different representations, combined with a judge that can identify **where** a candidate is likely overfitting — even when its reasoning reads as coherent.

B. Multimodal Candidate Generation

Candidate generation is performed via three foundation models across multiple inference configurations: **Gemini 3 Preview** in a high-reasoning setting (text, image, and code with tools), **GPT-5.2** in an x-high reasoning setting (text, image, code with and without tools, and a deep-think configuration), and **Claude Opus 4.5** in a long-context (120k) setting (text reasoning). Exact API parameters, tool schemas, and prompts are documented in the open-source implementation. In the final system, generators are grouped into three families — **Text**, **Image**, and **Code** — with multiple configurations within each family to encourage diversity. Table 1 shows the full candidate configuration.

Table 1. Candidate configuration: 29 generators grouped by family.

Family	Generator	Candidates
Text	Claude Opus 4.5 (text)	1
Text	Gemini 3 Preview (text)	1
Text	GPT-5.2 (text)	2
Text	GPT-5.2 (deep think)	4
Image	Gemini 3 Preview (image)	4
Image	GPT-5.2 (image)	6
Code	Gemini 3 Preview (code, tools)	2
Code	GPT-5.2 (code, tools)	9
Total		29

The text family contributes 8 candidates (including 4 deep-think runs), image contributes 10, and code contributes 11. Within each family, multiple runs of the same generator use the same prompt and API parameters; diversity arises from model sampling stochasticity.

Text methodology. Text candidates are generated by prompting a language model with a textual encoding of the ARC training pairs and test input. The model is asked to infer the transformation rule and output a completed test grid. The base prompt is intentionally minimal:

```

You are solving an ARC task. Each grid cell is an
integer
0-9 representing a color. Use the solved examples to
infer
the transformation and apply it to the test input.
{training and test examples}
  
```

```
Respond with an explanation of your thinking ... you
MUST
also respond with the completed output grid.
```

The prompt deliberately does not prescribe a fixed reasoning template, step-by-step plan, or output format, which reduces compliance overhead and empirically increases hypothesis diversity. The trade-off is noisier outputs that require tolerant parsing and validation to recover candidate grids (see Section VI for negative results on prescribed reasoning templates). Grids are encoded in CSV format, selected after benchmarking multiple representation formats; suboptimal format choices cost on the order of 10% lost performance.

“Deep think” variant. In addition to the standard text prompt, a “deep think” configuration allocates a larger test-time compute budget by explicitly encouraging GPT-5.2 to reason more extensively before committing to a final output — effectively trading tokens for deeper deliberation within a single response. This accounts for 4 of the 8 text candidates (Table 1).

Image methodology. Image candidates are generated by rendering the ARC training pairs and test input as a single annotated image alongside a textual instruction prompt. This provides the model with a pixel-space representation that can be advantageous for tasks where salient structure is more readily perceived visually than through a textual grid encoding. The image renderings are intentionally **imprecise** — slightly distorted rather than pixel-perfect grid reproductions. Pixel-perfect renderings underperformed in early experiments, possibly because models treated them as lossless encodings and fell back on cell-by-cell numerical reasoning rather than engaging the visual pattern recognition that makes image prompting valuable. The slight imprecision appears to encourage models to reason about shapes, symmetries, and spatial relationships at a higher level of abstraction. In the public evaluation analysis (Section IV-E), image prompting provides uniquely correct candidates on several instances.

Code methodology. Code candidates treat ARC solving as program synthesis: the model is asked to produce executable code that maps input grids to output grids. Two code-generation regimes are used. *Tool-integrated code generation* lets the model iteratively write code, execute it via a sandbox tool, inspect intermediate outputs, and refine the program across multiple tool calls;³ this regime often produces rich intermediate artifacts (program drafts, test harnesses, execution traces) that are later consumed by the holistic judge. *One-shot code generation* returns a complete program in a single response without iterative feedback — substantially cheaper but less robust on tasks that benefit from iterative debugging. The tool-integrated regime is the primary code strategy, contributing the majority of code candidates.

³The ARC Prize semi-private evaluation uses OpenAI’s zero-data-retention (ZDR) API mode, which disables tool calls. For the semi-private run, tool-integrated code candidates were replaced with one-shot code generation.

C. Context-Preserving Holistic Judging

Given 8–29 candidates per task, many are plausible and internally coherent. Worse, models tend to **cluster** around the same wrong interpretation on the hardest tasks. The key difficulty is identifying a rare candidate that is correct — or close enough that it can be repaired — without reducing everything to an overly lossy score. Three judging approaches were tested:

1. **Logic judge (failed):** Scoring candidates by whether their reasoning appears logically consistent fails because a candidate can be internally coherent yet overfit to training pairs or miss a latent concept introduced in the test case.
2. **Consistency judge (partial):** Selecting themes that repeat across candidates rewards the majority cluster, but breaking new ground requires elevating *divergent* hypotheses — the correct solution to an unsolved task is often not the modal answer.
3. **Holistic judge (final):** All traces are provided *together* in a single prompt, and the judge picks the top-2 most likely correct candidates. Three judges run in parallel and their picks are aggregated via weighted scoring. This works because having all context together beats abstracting traces into scores, letting the judge detect subtle but decisive differences between near-identical hypotheses.

The holistic judge can be understood as an **anti-consistency** mechanism. Self-consistency [9] selects the most common answer across samples — an effective strategy when the majority is likely correct. On ARC-AGI-2, the majority is often *wrong* on the hardest tasks (Section IV-F), making consistency-based selection a liability. The holistic judge inverts this: it is designed to identify a correct *minority* hypothesis against a confidently wrong majority, using full-trace comparison to distinguish genuine insight from plausible groupthink.

All three judges use **GPT-5.2** (x-high reasoning setting), the same model used for candidate generation but in a distinct role with a different prompt. A mixed-model ensemble (combining Opus, Gemini, and GPT-5.2) was tested, but three homogeneous GPT-5.2 judges outperformed the mixed configuration.

Judge prompt structure. The holistic judge prompt is assembled programmatically and follows this structure (condensed; full implementation in the open-source release):

```
Below is a problem attempted {N} times:
{training pairs + test input}
<SOLUTION 1 START>
<CONTENT>{full reasoning trace}</CONTENT>
<PREDICTED_GRID>{candidate output as
CSV}</PREDICTED_GRID>
<SOLUTION 1 STOP>
... (repeated for all N solutions) ...
Your task is to assess how well they've understood
the
problem. Often, new mechanics are introduced in the
test
example. Please output two solutions that represent
the
right mechanic for solving the problem.
```

Each candidate’s `CONTENT` block contains the full reasoning trace — for text/image candidates this is the model’s chain-of-thought response, and for code candidates it is the complete iterative tool-use trace including intermediate program drafts, execution outputs, and debugging steps. Candidates producing identical output grids are listed as separate solutions with separate traces, preserving the judge’s ability to assess reasoning quality even when outputs agree. The prompt does **not** instruct the judge to prefer majority or minority answers — it asks for a “meta-conclusion,” leaving the judge free to weigh agreement, reasoning quality, and novelty as it sees fit. The resulting prompt is intentionally large: on the order of **30k–80k input tokens**, making long-context frontier models a practical requirement for the judge step.

Synthesis. The holistic judge is also permitted to propose a **novel solution not identical to any candidate output**, effectively recombining correct subcomponents of multiple flawed candidates. This matters on tasks where no single candidate fully solves the problem but multiple candidates contain partial truths. Synthesized solutions are output as raw grids (not executable code), which means they do not benefit from programmatic verification and are susceptible to arithmetic or grid-construction errors.

Aggregation. Each judge outputs a ranked pair of solutions (first choice and second choice). The system assigns **2 points** to each judge’s first choice and **1 point** to each judge’s second choice, then sums points across judges for each distinct output grid. The two grids with the highest total score become the solver’s `pass@2` guesses. When judges agree on their top pick, that solution accumulates up to 6 points; when they disagree, the scoring naturally surfaces the most broadly supported candidates. A known limitation is that candidate traces are concatenated in a fixed order without shuffling between judge runs, which may interact with LLM position biases [16]; shuffling candidate order across runs is a natural improvement.

IV. EXPERIMENTS AND RESULTS

A. Evaluation Protocol

ARC-AGI-2 provides public, semi-private, and private evaluation sets, all calibrated and scored under **pass@2** [1]. ARC Prize Verified results are reported on the semi-private evaluation set via an official verification process and leaderboard. The solver was iteratively developed using the 1,000-task training set and the 120-task public evaluation split; the semi-private evaluation was run on held-out tasks unseen during development, and the ~3 percentage-point gap between public and semi-private scores suggests limited overfitting.

B. Headline Results

The solver was submitted to the ARC Prize foundation on December 15, 2025, with official results announced on February 3, 2026. The leaderboard snapshot in Table 2 reflects the state at the time of announcement.

On the **semi-private evaluation set**, the solver achieves **72.9% solved** at **\$38.99/task** – the highest score on the ARC Prize Verified leaderboard at the time of writing. On the **public evaluation set**, it achieves **76.11% solved** at **\$19.69/task** (self-measured). For the per-instance analyses below, the public evaluation split contains 120 task IDs with 167 test instances (75 tasks with 1 test instance, 43 with 2, and 2 with 3).

The ~3 percentage-point gap between public (76.11%) and semi-private (72.9%) likely reflects three factors: (i) natural generalization loss to a held-out task distribution, (ii) the semi-private verification uses OpenAI’s zero-data-retention (ZDR) API mode, which disables function/tool calling, and (iii) the semi-private run coincided with a period of known instability in OpenAI’s API. Under ZDR, the tool-integrated code generation candidates (Section III-B) were replaced with one-shot code generation without iterative sandbox execution. Code candidates were still produced, but without the iterative debugging loop that makes tool-integrated generation more robust on complex tasks. Since tool-integrated code generation accounts for the bulk of the Code family’s cost (Table 7), the ZDR constraint both reduced accuracy and changed the cost profile of the semi-private run relative to the public run.

Table 2. Leaderboard snapshot and reference systems.⁴

Semi-private results are as reported on the ARC Prize Verified leaderboard at the time of the official results announcement (February 3, 2026); the public-evaluation row is self-measured.

AI System	Author	ARC-AGI-2	Cost/Task	Comment
Human Panel	Human	100.00%	\$17.00	At least two humans out of ~400 solved it Semi-private (official) Public eval
This paper	Johan Land	72.90%	\$38.99	
This paper	Johan Land	76.11%	\$19.69	
GPT-5.2 Pro (High)	OpenAI	54.20%	\$15.72	
Gemini 3 Pro (Refine.)	Poetiq	54.00%	\$30.57	
GPT-5.2 (X-High)	OpenAI	52.90%	\$1.90	
Gemini 3 Deep Think (Preview)	Google	45.10%	\$77.16	
GPT-5.2 (High)	OpenAI	43.30%	\$1.39	
GPT-5.2 Pro (Medium)	OpenAI	38.50%	\$8.99	
Opus 4.5 (Thinking, 64K)	Anthropic	37.60%	\$2.40	
Gemini 3 Flash Preview (High)	Google	33.60%	\$0.23	

The system achieves +18.7 percentage points over the strongest commercial baselines at higher cost, reflecting additional test-time compute spent on multi-candidate search.

⁴<https://arcprize.org/leaderboard>

The remaining gap to the human panel (100.0% at \$17/task) indicates substantial headroom. As the leaderboard evolves rapidly, the contribution is the **architectural pattern** rather than the specific accuracy numbers.

C. Efficiency

ARC-AGI-2 explicitly evaluates efficiency [1]. The roughly 2x cost difference between the semi-private and public runs (\$38.99 vs. \$19.69) is largely attributable to API-level unreliability: on the public-eval run, only 2,216 of 14,106 GPT-5.2 API attempts succeeded (84% failure rate due to rate limits, timeouts, and server errors). The public-eval cost of \$19.69/task is a more representative measure; at this cost, the solver is comparable to GPT-5.2 Pro (\$15.72/task) while achieving a +21.9 percentage-point accuracy gain, and is both cheaper and more accurate than Gemini 3 Pro (\$30.57/task at 54.0%). A detailed cost breakdown is provided in Section V.

D. Modality Contribution

The final modality mix was selected based on both raw solve contribution and diversity contribution (uniquely solved tasks that other modalities fail). GPT dominates code generation, Gemini adds meaningful diversity, and Gemini/GPT image reasoning behave differently enough to be complementary; Opus remains unusually strong for end-to-end text reasoning and is the sole solver for several text-only successes. Over the 167 public-evaluation test instances, the final system solves $128/167 = 76.65\%$ at the instance level (pass@2), while the candidate pool contains at least one correct output for $144/167 = 86.23\%$ (candidate-oracle accuracy). The remaining 39 instances decompose into 21 generation failures where no candidate in the pool is correct (with full 29-candidate coverage), 1 early-stopping failure, and 17 selection failures where at least one correct candidate exists but is not chosen by the judge. Generation failures tend to involve long chains of dependent reasoning steps, while selection failures cluster around tasks where the test case introduces a mechanic not fully disambiguated by training examples. One additional instance (21897d95:2) is solved by judge synthesis despite **zero** candidates matching ground truth; this occurs via judge synthesis rather than candidate selection.

E. Modality Complementarity (Public Eval)

To quantify complementarity between candidate-generation methodologies, I evaluate each candidate output against ground truth and record a per-instance correctness matrix. To avoid conflating uniqueness with conditional execution (the system uses adaptive early stopping for easy tasks), the analysis is restricted to the 130 instances with complete candidate coverage (all 29 candidates generated).

Table 3. Pairwise non-overlap between modality families ($n = 130$). Each cell reads “row solves, column does not”: the entry counts instances with at least one PASS in the row family and zero PASS in the column family. The table is not symmetric.

Each family covers a non-trivial set of instances that are not covered by at least one other family. At the exclusive level,

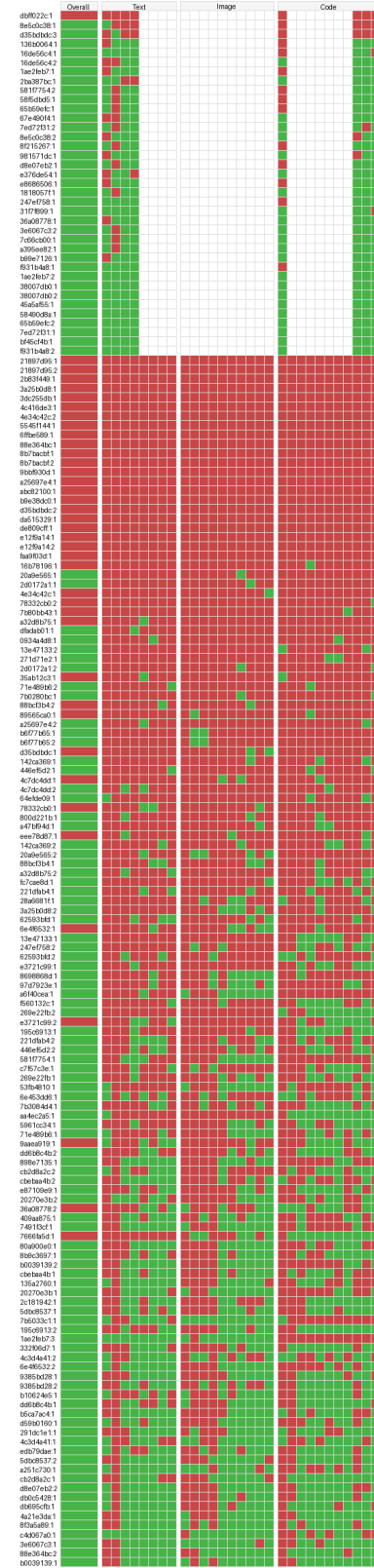


Fig. 3. Methodology matrix over public evaluation instances. Green = correct candidate; red = incorrect candidate; white = candidate not produced due to early stopping.

	Text	Image	Code
Text	NA	13	7
Image	17	NA	11
Code	18	18	NA

2 instances are solved only by Text, 6 only by Image, and 7 only by Code. This confirms that modalities function as distinct search operators rather than redundant copies of the same reasoning process, and supports treating each modality as a separate exploration channel in the candidate-generation phase.

F. Judging and Synthesis (Public Eval)

Holistic judging (excluding synthesis) yielded a net uplift of **+7 solved instances** relative to a majority-vote baseline that selects the most common candidate output. All 7 are minority recoveries – cases where the correct answer was not the most frequent candidate output and the holistic judge identified it by reasoning over full traces rather than counting votes. This directly validates the anti-consistency motivation: on these tasks the majority cluster was wrong, and the judge’s ability to read and compare reasoning traces was the deciding factor. One illustrative case is `dfadab01:1`, where 12 candidates converge on one incorrect output and 8 on another, but the judge identifies the lone correct candidate. The judge rationale shows explicit anti-consistency reasoning:

```
Most candidate solvers correctly identify the core
stamp
mechanic ... The only remaining ambiguity is edge
handling
(not clearly disambiguated by the training set):
- Some solvers assume a stamp must fit fully.
- Others assume stamps are clipped at the border.
So the two most plausible outputs (same mechanic,
differing only in border handling)
```

Rather than committing both guesses to the majority interpretation, the judge uses the `pass@2` format to hedge across the two edge-handling interpretations — reasoning that majority voting cannot perform.

Synthesis – where the judge produces a novel output not identical to any candidate – yielded **+1 additional solved instance** (21897d95:2), a task where none of the 29 candidates were correct but the judge recombined partial insights (correct room parsing from some candidates, correct arrow semantics from others) into a novel correct output.

Although the measured count is only one instance, this case is qualitatively different from the +7 minority recoveries above. Holistic selection succeeds by identifying a rare correct candidate that already exists in the pool; synthesis succeeds when **no correct candidate exists at all**, but complementary partial truths are distributed across multiple failed candidates. In that sense, 21897d95:2 should be read as a proof-of-possibility for **compositional repair**: the architecture can sometimes do more than rank hypotheses and can instead construct a correct answer by recombining partial insights.

V. ABLATION STUDIES

Rigorous component attribution would require multiple full-pipeline reruns, each costing about \$2,400 in API spend (Table 6). This section therefore relies on **post-hoc analysis of the single public evaluation run**, which is informative but cannot capture all interaction effects. Unless otherwise stated, “solved” refers to `pass@2` at the **test-instance** level on the 167-instance public evaluation split.

A. Measured Ablations

Table 5. Measured judge ablations on the public evaluation run. Reported deltas are net solved-instance uplifts attributable to the indicated component.

Component	Ablation / control condition	Net uplift (solved instances)
Holistic selection	Holistic judge vs majority-vote baseline (synthesis disabled in both)	+7 (all minority recoveries)
Judge synthesis	Synthesis enabled vs disabled (holistic selection held fixed)	+1

The +7 holistic-selection uplift consists entirely of minority recoveries where the correct answer was not the most frequent candidate output. Synthesis adds +1 more instance by recombining complementary partial insights when no candidate is fully correct. While the measured synthesis count is small in this run, the mechanism is most relevant on harder tasks where no single candidate fully solves the problem.

B. Cost Attribution

Table 6. Cost attribution per test instance on the public evaluation run (n = 167).

Phase	Total (\$)	Avg \$/instance	% of total
Candidate generation	2081.37	12.46	87.1%
Judging	308.91	1.85	12.9%
Total	2390.28	14.31	100%

Table 7. Candidate generation cost by modality family.

Modality family	Total (\$)	Avg \$/instance	% of generation cost
Text (incl. deep think)	597.70	3.58	28.7%
Image	467.10	2.80	22.5%
Code	1016.56	6.09	48.9%

Candidate generation dominates at 87% of spend; code is the most expensive family because of iterative sandbox execution. Judging contributes +7 solved instances from holistic selection and +1 from synthesis at only 13% of total cost.

Table 8. Post-hoc marginal oracle coverage on the complete-coverage subset (n = 130). This oracle-only analysis uses the 130 instances where all 29 candidates were produced. It estimates lower-bound generation gains under a

Candidate budget	Families included	Oracle-solvable instances	Marginal gain vs previous stage
8 candidates	Text only	84 / 130 (64.6%)	–
18 candidates	Text + Image	101 / 130 (77.7%)	+17
29 candidates	Text + Image + Code	108 / 130 (83.1%)	+7

hypothetical staged family expansion; judges are not re-run on reduced pools.

The main takeaway is modest but useful: on the hard-instance subset, additional candidate families are **not equally valuable**, and staged expansion can recover meaningful oracle coverage beyond a cheap base. Because this is a post-hoc oracle analysis under one ordering, it should be read only as evidence for **modality-aware adaptive routing**. This complements the early-stopping result: 37/167 instances terminated after the initial 8-candidate probe, and 36 of those 37 were solved correctly.

C. Unperformed Ablations

The following ablations would strengthen the paper’s claims but have not been run due to cost constraints (~\$2,400 per full run):

- **End-to-end modality removal:** run the full pipeline with one modality family removed and re-run judging on the reduced candidate pool, capturing interaction effects not visible in oracle-level analysis.
- **Matched candidate budget scaling:** rerun the full pipeline at multiple budget points and with multiple routing orders to estimate marginal returns under controlled conditions. Table 8 provides only a post-hoc oracle lower bound, not a full end-to-end scaling curve.
- **Trace content ablation:** compare judge accuracy with full reasoning traces vs output grids only, to quantify whether trace content actually helps selection or whether the judge primarily relies on output comparison.

VI. NEGATIVE RESULTS

Several development ideas were discarded because they reduced diversity or increased brittleness. Two findings stand out for their direct impact on the final system design.

Prescriptive prompting degrades performance. Reasoning templates, structured output requirements, chain-of-thought scaffolding, and domain-specific heuristics consistently hurt performance on the hardest tasks. The mechanism appears to be a **compliance tax on reasoning**: when told exactly *how* to think, models spend more of their test-time compute budget following the template and less on exploring unconventional hypotheses. On easy tasks this overhead is often harmless, but on hard tasks it suppresses the creative leaps needed to escape the main wrong-answer cluster. Prescriptive prompts also interact badly with diversity: when all candidates

follow the same template, they are more likely to converge on the same flawed interpretation. The final system therefore uses deliberately minimal prompts with tolerant downstream parsing.

Hint generation collapsed diversity. A two-stage approach – generate a hint, then solve conditioned on that hint, structurally similar to Self-Refine [13] and Reflexion [14] – was tested and discarded. The hint stage consistently narrowed candidate diversity into a smaller hypothesis space, which is counterproductive when the correct solution may require an unconventional reasoning path. Similarly, staged decomposition pipelines often suffered brittle handoffs, regressing candidate outputs toward the mean rather than broadening the search.

Several other approaches were also discarded: strict output schemas, synthetic code-data augmentation, and weaker generator configurations that contributed little unique coverage. The synthetic augmentations were especially weak because surface-level transforms did not create new structural challenges, and geometric transforms could even break task semantics on orientation-dependent problems.

VII. DISCUSSION AND CONCLUSION

This paper demonstrates that strong ARC-AGI-2 performance can be achieved by treating **modalities as search operators** and selecting with holistic judging over full traces. At the time of writing, the approach achieves 72.9% on the ARC Prize Verified leaderboard, exceeding the best standalone frontier models – GPT-5.2 Pro (54.2%) and Gemini 3 Pro (54.0%) – by +18.7 points. Within ARC-AGI-2, this suggests that orchestrating diverse reasoning modalities is more effective than scaling any single model alone.

The system’s core contributions are architectural rather than model-specific. First, multi-modal candidate generation exploits the fact that text, image, and code representations activate different reasoning pathways, and each modality family contributes unique solves (Table 3). Second, holistic judging over full reasoning traces recovers minority-correct solutions that majority voting discards, yielding +7 instances of uplift. Third, judge synthesis can produce a novel correct output from complementary partial insights when no candidate is fully correct. Although synthesis appears only once in this run, that case is qualitatively important: it shows the architecture can sometimes **recombine partial evidence into a new correct solution**. These mechanisms are complementary: diverse generation expands the hypothesis space, while context-preserving selection navigates it and, in rare cases, repairs it compositionally.

Any claim of broader generality should be stated cautiously. The current evidence comes only from ARC-AGI-2, so cross-benchmark transfer is unproven. The most plausibly portable components are the high-level ideas: diverse hypothesis generation across representations, joint comparison of full candidate traces, and selective synthesis when partial truths are distributed across candidates. ARC-specific elements such as the

grid encodings, the image-rendering setup, the 29-candidate budget, and the pass@2 aggregation rule should be viewed as task-tuned choices rather than universal prescriptions.

A. Limitations

Cost and practical scalability. The system spends \$19.69–\$38.99 per task – far more than a single model call. ARC Prize explicitly treats efficiency as a first-class metric, and the current 29-candidate, three-judge pipeline is effective but wasteful; a practical system would need adaptive routing that allocates expensive modalities only when uncertainty is high.

Single-run results. The headline numbers (76.11% public, 72.9% semi-private) each come from a single evaluation run. Because outputs are stochastic, results will vary across runs; no confidence intervals are reported because repeated full-pipeline runs were not performed.

No learning across tasks. The system treats each task independently; no information is carried from one task to the next.

Narrow evaluation domain. The system has been evaluated only on ARC-AGI-2. Whether the architectural pattern generalizes to other benchmarks or broader AI tasks is unknown.

Reproducibility fragility. The system depends on proprietary model snapshots (GPT-5.2, Gemini 3 Preview, Opus 4.5) and vendor-specific reasoning settings that may change or become unavailable over time. Source code and complete public-run artifacts are released, but results may not be exactly reproducible if the underlying model versions drift.⁵

B. Future Directions

Two directions appear most promising. **Adaptive routing** would allocate expensive modalities only when early-stage candidates show low agreement, reducing cost without sacrificing accuracy on hard tasks. The current early-stopping heuristic already works well on easy tasks, and the post-hoc analysis in Section V suggests that later candidate families should be added in stages rather than treated as equally valuable. **Synthesis gating** would trigger judge synthesis only when candidate agreement is low or when multiple candidates contain complementary partial solutions, potentially amplifying the +1 instance uplift observed here.

ACKNOWLEDGMENT

Thank you to the ARC-AGI Discord community for valuable discussion and shared insights throughout the development of this work, and to Greg Kamradt at ARC Prize for conducting the official semi-private evaluation.

REFERENCES

- [1] F. Chollet, M. Knoop, G. Kamradt, and B. Landers, “ARC Prize 2024: Technical report,” *arXiv preprint arXiv:2412.04604*, 2024.
- [2] J. Land, “Modality-driven search with holistic trace judging for ARC-AGI-2,” *arXiv preprint*, 2026.
- [3] F. Chollet, “On the measure of intelligence,” *arXiv preprint arXiv:1911.01547*, 2019.
- [4] S. Ferré, “First steps of an approach to the ARC challenge based on descriptive grid models and the minimum description length principle,” *arXiv preprint arXiv:2112.00848*, 2021.
- [5] W.-D. Li *et al.*, “Combining induction and transduction for abstract reasoning,” in *International conference on learning representations*, 2025.
- [6] E. Akyürek *et al.*, “The surprising effectiveness of test-time training for few-shot learning,” in *Proceedings of the 42nd international conference on machine learning*, in PMLR, vol. 267. 2025, pp. 942–963.
- [7] M. V. Macfarlane and C. Bonnet, “Searching latent program spaces,” in *Advances in neural information processing systems*, 2025.
- [8] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [9] X. Wang *et al.*, “Self-consistency improves chain of thought reasoning in language models,” in *International conference on learning representations*, 2023.
- [10] S. Yao *et al.*, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in neural information processing systems*, 2023.
- [11] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” *International Conference on Learning Representations*, 2023.
- [12] L. Gao *et al.*, “PAL: Program-aided language models,” *International Conference on Machine Learning*, 2023.
- [13] A. Madaan *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [14] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [15] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling LLM test-time compute optimally can be more effective than scaling model parameters,” in *International conference on learning representations*, 2025.
- [16] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.

⁵Source code: <https://github.com/beetree/ARC-AGI>. Public-evaluation run data: <https://www.kaggle.com/code/johanland/johan-land-solver-v7-public/comments?scriptVersionId=290052212>.